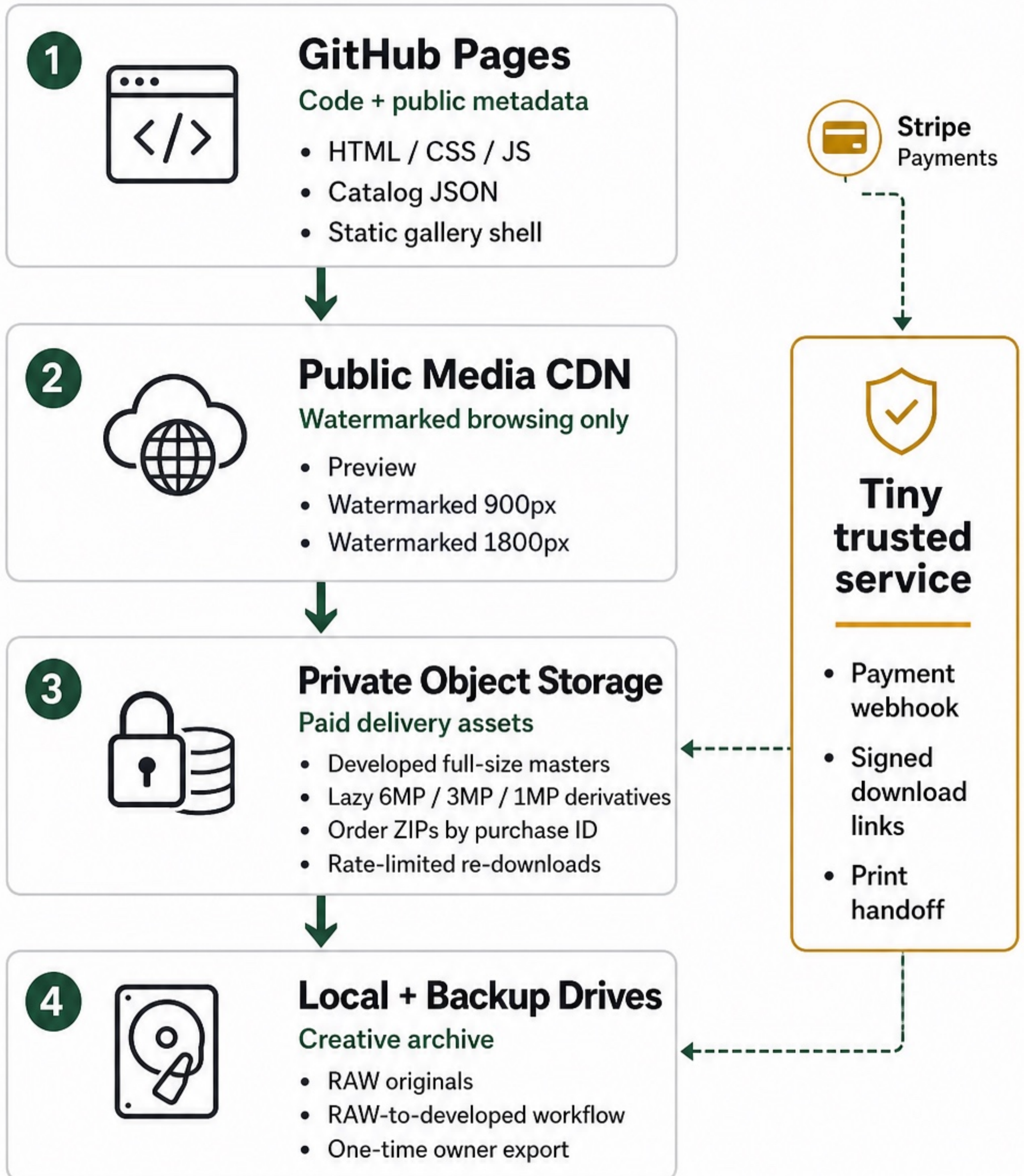


PhotosByElie:

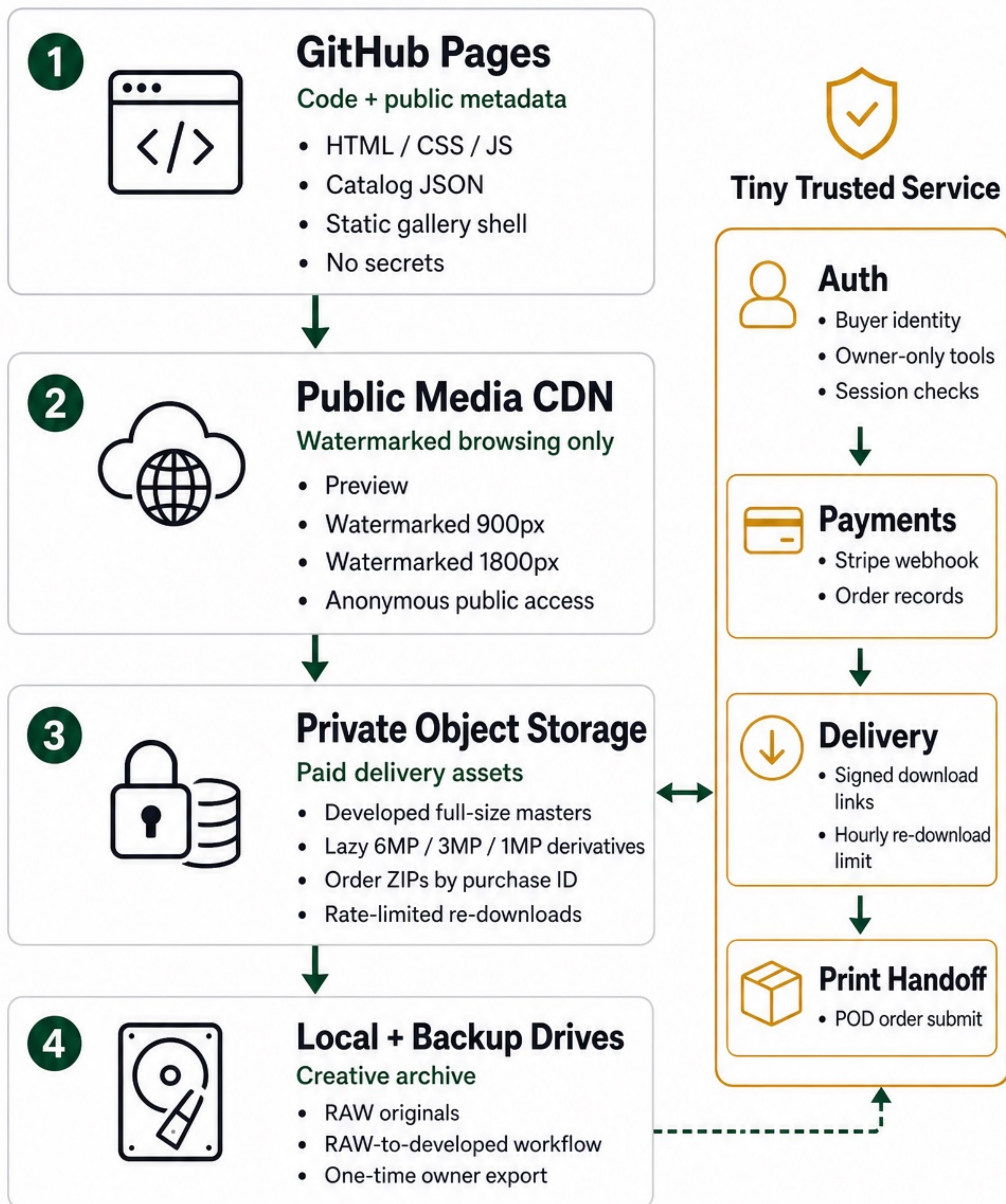
Static-First Media Architecture



Public space gets **watermarked images only**.
RAW files stay local.

PhotosByElie:

Static-First Media Architecture



Public space gets **watermarked images only**.
Auth protects **paid delivery** and **owner tools**.
RAW files stay local.

PhotosByElie: Starting Cloud Budget

Static storefront, watermarked public media, private paid delivery assets.



Storage
\$3 / month

200 GB Cloudflare R2
at about \$0.015 / GB-month



Trusted Service
\$0-\$5 / month

Auth checks, Stripe webhook,
signed links, hourly rate limit



Payments
\$0 fixed

Stripe transaction fees only



Lean start: \$3-\$5 / month
+ Stripe fees



Comfortable start: \$8-\$12 / month
+ Stripe fees

Sale Amount	Est. Stripe Fee
\$45 sale	about \$1.61
\$100 sale	about \$3.20
\$250 sale	about \$7.55



Assumes US domestic
card rate: 2.9% + 30c

High-Level Architecture



GitHub Pages
Static site hosting
(code + metadata)



Public R2:
watermarked images
Small/medium images
for browsing and sharing



Private R2:
masters + order ZIPs
Full-size masters,
on-demand sizes,
and order ZIPs



Worker:
auth + delivery
Auth checks, signed
URLs, hourly rate limit,
secure delivery



Budget target: under \$10/month to start, before payment processing.

Cloudflare R2 Storage

Object storage for public watermarked browsing images and private paid-delivery assets.

ROLE IN PLAN

One storage platform:
public bucket/path for
watermarked media,
private bucket/path for
developed masters and
order ZIPs.

- ✗ RAW originals
- ✗ clean public freebies
- ✗ secrets in GitHub

SIGNUP / SETUP TODO

- ☐ Create or sign in to a Cloudflare account.
- ☐ Dashboard: Storage & databases > R2 > Overview; add the R2 subscription and complete checkout.
- ☐ Create public and private buckets or prefixes, for example photosbyelie-public and photosbyelie-private.
- ☐ Keep private assets non-public. Public space gets watermarked images only.
- ☐ Choose access method: dashboard for tests, S3-compatible tools for bulk upload, Worker bindings for the app.
- ☐ Record account ID, bucket names, endpoint, and API-token location in the password manager.
- ☐ Upload one watermarked 900/1800 pair and verify the public URL works.

DECISIONS TO MAKE LATER

- bucket names
- custom media domain
- lifecycle rules
- backup/export policy

SERVICE CHECKLIST

Cloudflare Workers + KV

Tiny trusted service for secrets, order lookups, signed links, and hourly download throttles.

ROLE IN PLAN

The static site calls the Worker only when the browser needs trusted behavior that cannot live in JavaScript.

Do not put here:

- ✗ RAW originals
- ✗ clean public freebies
- ✗ secrets in GitHub

SIGNUP / SETUP TODO

- ☐ Install Node.js and create a Worker project with `npm create cloudflare@latest`.
- ☐ Log in with Wrangler and deploy a hello-world Worker first.
- ☐ Bind the private R2 bucket to the Worker; optionally bind the public bucket too.
- ☐ Create a KV namespace or D1 database for orders, rate-limit timestamps, and delivery metadata.
- ☐ Add routes: `/stripe-webhook`, `/order/:id`, `/download/:orderId`, `/signed-url`, and `/health`.
- ☐ Store Stripe secrets and signing secrets with `wrangler secret put`, never in GitHub.
- ☐ Test locally with `wrangler dev`, then deploy and hit the health endpoint.

DECISIONS TO MAKE LATER

KV vs D1

custom API domain

log retention

retry policy

SERVICE CHECKLIST

Cloudflare Access / Auth

Owner-only protection first; buyer identity can start as order-token plus email link.

ROLE IN PLAN

Auth protects owner tools and paid delivery. It should not block anonymous public browsing.

Do not put here:

- ✗ RAW originals
- ✗ clean public freebies
- ✗ secrets in GitHub

SIGNUP / SETUP TODO

- ☐ Create a Cloudflare Zero Trust account and team domain.
- ☐ Choose the first identity method: email one-time PIN is enough for owner-only v1.
- ☐ Add an Access application for owner/admin routes or a private owner subdomain.
- ☐ Create an allow policy for the owner email address only.
- ☐ Keep the public gallery outside Access so buyers can browse anonymously.
- ☐ For buyer downloads, start with purchase-order token plus email verification; add buyer accounts later only if needed.
- ☐ Test allowed owner access, denied non-owner access, and a normal anonymous gallery visit.

DECISIONS TO MAKE LATER

buyer accounts?

owner subdomain

session length

email provider

SERVICE CHECKLIST

Stripe Payments

Payment collection, order confirmation, and webhook trigger for delivery ZIP creation.

ROLE IN PLAN

Stripe owns card handling. The Worker listens for paid events and unlocks delivery.

Do not put here:

- ✗ RAW originals
- ✗ clean public freebies
- ✗ secrets in GitHub

SIGNUP / SETUP TODO

- ☐ Create a Stripe account and complete business, identity, tax, and bank setup.
- ☐ Start in test mode; do not switch live until delivery and refunds are tested.
- ☐ Choose v1 checkout path: Payment Links for simple fixed items, or Checkout Sessions for a custom basket.
- ☐ Create products/prices for license sizes or print products as needed.
- ☐ Add a webhook endpoint pointing to the Worker, for example /stripe-webhook.
- ☐ Store the webhook signing secret in Cloudflare Worker secrets.
- ☐ Run a test purchase and confirm an order record, delivery ZIP path, and signed download link are created.

DECISIONS TO MAKE LATER

Payment Links vs Checkout

tax settings

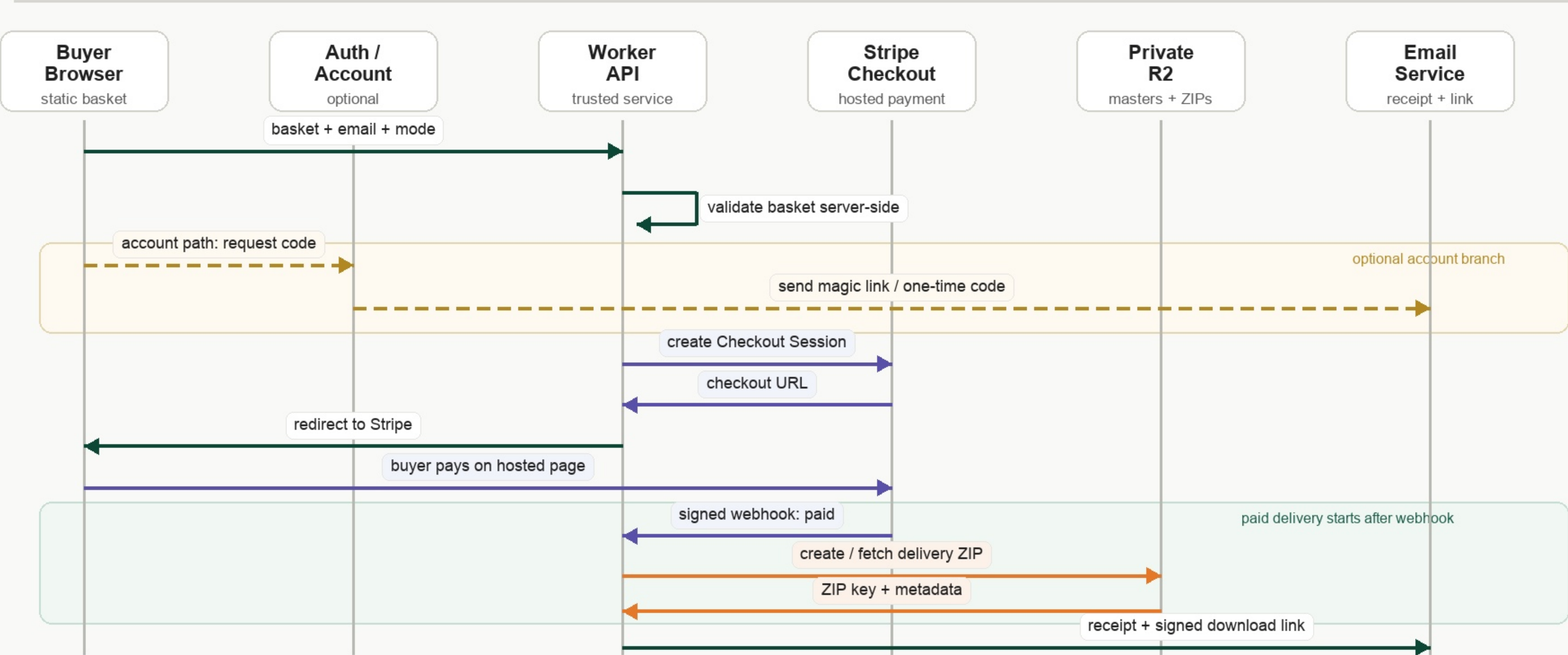
refund policy

receipt wording

MESSAGE SEQUENCE CHART

Basket Checkout: Guest First, Account Optional

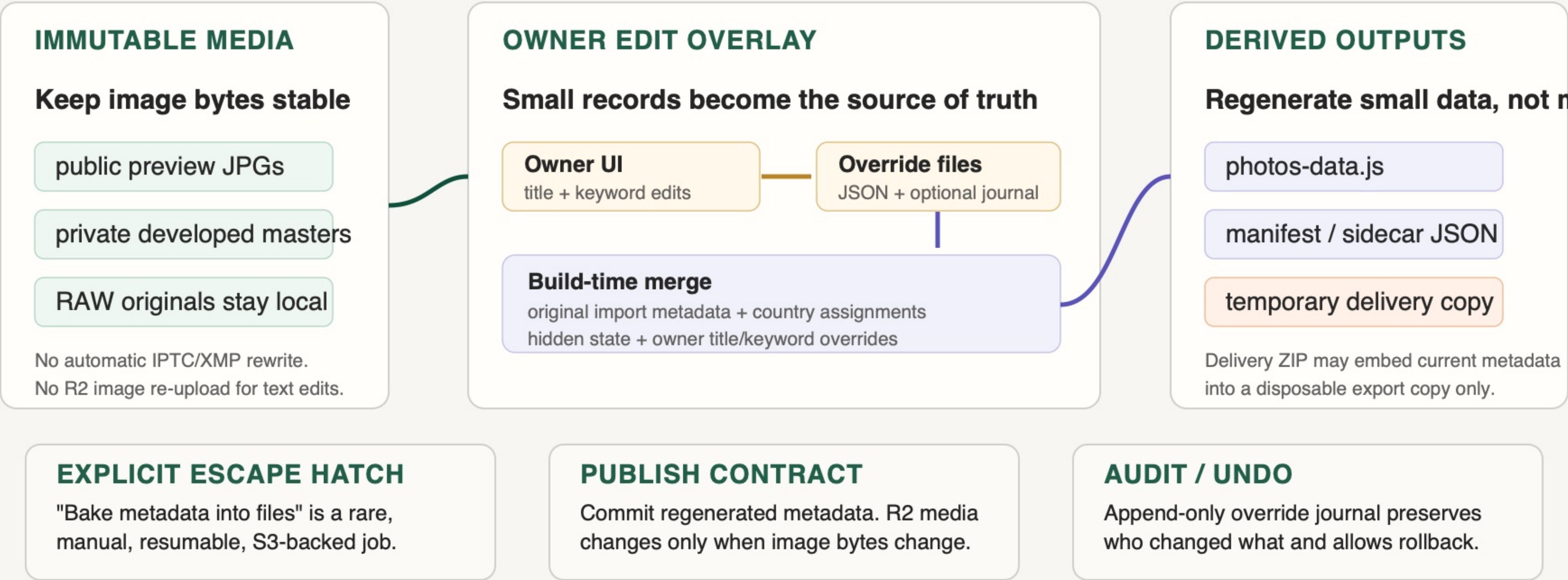
Time flows down. Horizontal arrows show who sends what, what triggers it, and what state changes.



RESULT Paid order record + private ZIP + signed link. Browser proposes; Worker verifies after Stripe confirms.

Non-Destructive Metadata Overrides

Owner edits behave like RAW editor adjustments: reversible metadata overlays, not media rewrites.



RESULT Fast owner edits, stable R2 objects, reversible metadata, and delivery files that reflect the latest catalog at ZIP time.